# DIPLOMATIC FLEET

## Documentación técnica y funcional de la primera entrega

Sistema de gestión de servicios vehiculares institucionales

Daniela Largo Corredor

Xiomara Lobatón Galindo

Bryan Steven Pardo Núñez

María Camila Pacheco Morales

Politécnico Grancolombiano

Práctica Aplicada a Sistemas

EDILBERTO TORRES ORTIZ

Bogotá, Colombia

Octubre de 2026

**Resumen**

Este documento consolida la documentación técnica y funcional de la primera entrega de Diplomatic Fleet. La versión presentada sustituye la demo aislada basada únicamente en datos locales por una aplicación web con frontend en Vue.js y TypeScript, backend en Kotlin con Spring Boot, persistencia en PostgreSQL y migraciones mediante Flyway. Los módulos de clientes institucionales y vehículos funcionan de extremo a extremo, mientras que Inicio, Documentos, Mantenimientos, Disponibilidad, Solicitudes, Validación de conductor, Cotizaciones y Usuarios y roles se encuentran disponibles como vistas navegables preparadas para su integración progresiva. La entrega incluye autenticación temporal, navegación compartida, validaciones, manejo uniforme de errores, documentación KDoc, documentación Markdown, pruebas de compilación y un flujo de trabajo basado en ramas, commits y Pull Requests revisados.

**Palabras clave:** Kotlin, Spring Boot, Vue.js, PostgreSQL, API REST, gestión vehicular, clientes institucionales, arquitectura por capas.

**Tabla de contenido**

1. Introducción 5

2. Objetivos 5

3. Alcance y estado de la primera entrega 6

4. Metodología y control de versiones 7

5. Arquitectura del sistema 8

6. Arquitectura del código fuente 14

7. Modelo de datos 15

8. Requisitos funcionales e historias de usuario 16

9. Módulos implementados 17

10. Módulos navegables en fase de integración 19

11. Seguridad y control de acceso 19

12. Validaciones y manejo de errores 20

13. Pruebas y evidencias de calidad 20

14. Instalación y ejecución local 21

15. Traslado del sistema a otro equipo 22

16. Resultados de la primera entrega 23

17. Limitaciones y trabajo futuro 23

18. Conclusiones 24

Referencias 24

Apéndice A. Credenciales y datos de demostración 25

Apéndice B. Lista de verificación para la exposición 25

Apéndice C. Control documental 26

# 1. Introducción

Diplomatic Fleet es un sistema web académico orientado a la administración de servicios vehiculares para embajadas, consulados, organismos internacionales y otros clientes institucionales. La solución busca centralizar el registro de entidades, vehículos, documentación, mantenimientos, disponibilidad, solicitudes, validaciones de conductor, cotizaciones, usuarios y roles.

La primera entrega demuestra una arquitectura funcional de extremo a extremo en dos módulos prioritarios. Clientes institucionales y Vehículos intercambian información entre Vue.js y una API REST protegida en Spring Boot, que persiste los datos en PostgreSQL. Los demás módulos cuentan con rutas, navegación común y vistas demostrativas con una identidad visual unificada, de modo que el proyecto puede continuar de forma incremental sin rehacer la estructura base.

Este documento actualiza el Manual integral corregido de Diplomatic Fleet y reemplaza afirmaciones que correspondían exclusivamente a la demo pública anterior. En el estado actual, la aplicación local ya no depende de localStorage para los módulos funcionales implementados. La persistencia real se verifica en PostgreSQL y el frontend utiliza servicios TypeScript para consumir la API.

# 2. Objetivos

## 2.1 Objetivo general

Desarrollar y documentar una primera versión funcional de Diplomatic Fleet que permita gestionar clientes institucionales y vehículos mediante una aplicación web conectada con una API REST y una base de datos relacional.

### 2.2 Objetivos específicos

• Implementar una arquitectura por capas que separe presentación, lógica de negocio, acceso a datos y persistencia.

• Crear una interfaz web en Vue.js y TypeScript con navegación común, diseño adaptable y mensajes claros.

• Implementar servicios REST en Kotlin y Spring Boot para clientes institucionales y vehículos.

• Garantizar integridad mediante restricciones, validaciones de negocio, índices únicos y migraciones Flyway.

• Aplicar autenticación temporal para la demostración y evitar el almacenamiento de secretos en el repositorio.

• Documentar el código, la arquitectura, las pruebas, los endpoints y el proceso de instalación.

• Mantener trazabilidad mediante Git, ramas de funcionalidad, commits con autoría individual y Pull Requests revisados.

## 3. Alcance y estado de la primera entrega

El alcance actual combina módulos completamente funcionales con otros preparados como prototipo navegable. Esta distinción es importante para presentar con transparencia el estado real del producto.

| Módulo | Estado | Alcance verificado |
|---|---|---|
| Acceso | Funcional temporal | Valida correo autorizado, contraseña técnica, aceptación legal y navegación al panel. |
| Clientes institucionales | Funcional | Registro, listado, búsqueda, consulta, actualización de estado y persistencia PostgreSQL. |

| | | |
|---|---|---|
| Vehículos | Funcional | Registro, listado, búsqueda, consulta, indicadores y persistencia PostgreSQL. |
| Inicio | Navegable | Panel general y acceso a los módulos. |
| Documentos | Prototipo navegable | Vista y datos demostrativos preparados para integración. |
| Mantenimientos | Prototipo navegable | Vista y datos demostrativos preparados para integración. |
| Disponibilidad | Prototipo navegable | Vista de consulta preparada para lógica de periodos y estados. |
| Solicitudes | Prototipo navegable | Vista preparada para el flujo institucional. |
| Validación de conductor | Prototipo navegable | Vista preparada para validación documental. |
| Cotizaciones | Prototipo navegable | Vista preparada para cálculo y seguimiento. |
| Usuarios y roles | Prototipo navegable | Vista preparada para seguridad basada en roles. |

La aplicación presenta los módulos pendientes como parte del roadmap, sin afirmar que sus reglas de negocio o su persistencia estén terminadas. El patrón aplicado en Clientes y Vehículos constituye la referencia para las siguientes integraciones.

## 4. Metodología y control de versiones

El equipo aplicó un desarrollo incremental con Git y GitHub. Cada bloque se desarrolló en una rama específica, se validó localmente, se confirmó mediante commits descriptivos y se sometió a revisión mediante Pull Request. Las revisiones incluyeron comentarios técnicos en Files changed, marcación de archivos revisados, aprobación y posterior fusión con main.

| **Responsable** | **Aportes principales verificados** |
|---|---|
| Daniela Largo Corredor | Servicio y API REST de vehículos, seguridad provisional, manejo global de excepciones y validaciones del módulo de flota. |
| Xiomara Lobatón Galindo | Integración frontend de vehículos, navegación completa, módulos demostrativos, API y frontend de clientes institucionales, documentación del módulo. |

| | |
|---|---|
| Bryan Steven Pardo Núñez | Arquitectura base, coordinación del repositorio, integración y validación de la entrega. |
| María Camila Pacheco Morales | Estructura visual y componentes del inicio de sesión, validaciones y recursos de interfaz. |

Los commits se atribuyeron al integrante responsable mediante la configuración local de user.name y user.email. Esta práctica permitió que el historial reflejara la participación individual sin usar coautoría cuando el criterio académico requería autor principal por contribución.

## 5. Arquitectura del sistema

La arquitectura separa el frontend, la API, la lógica de negocio y la persistencia. Vue.js ejecuta la interfaz en el navegador y envía solicitudes HTTP con JSON. Spring Security protege los endpoints con autenticación HTTP Basic temporal. Los controladores delegan en servicios transaccionales, los repositorios emplean Spring Data JPA y PostgreSQL conserva la información. Flyway versiona el esquema.

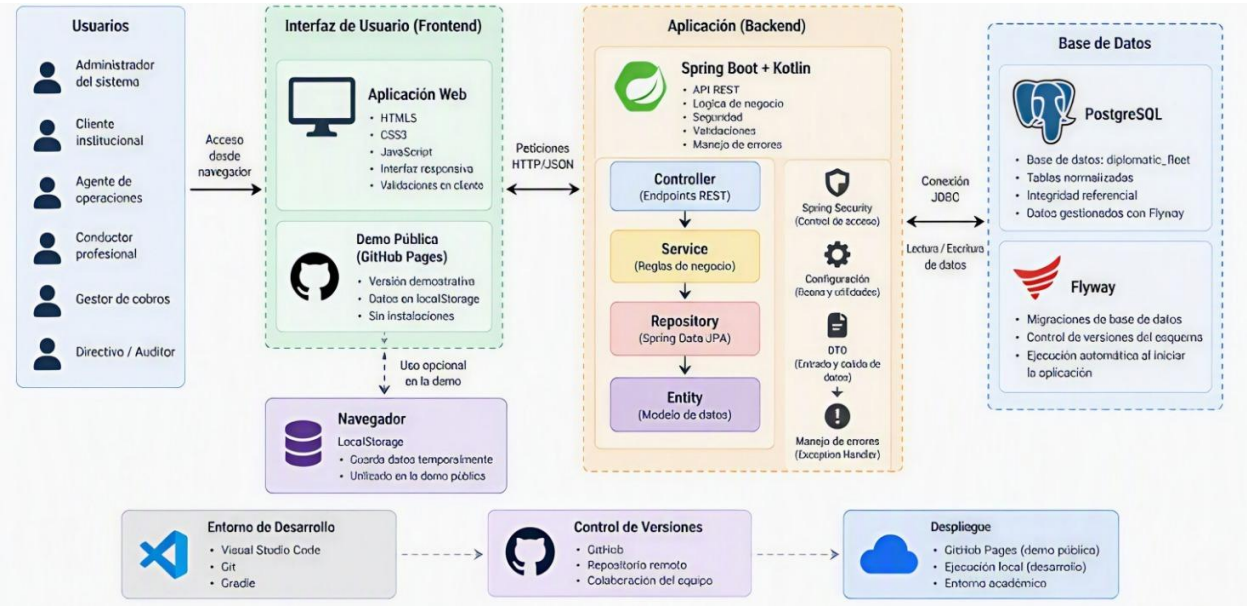



**Figura 1. Arquitectura general del sistema Diplomatic Fleet.**

*Nota. Elaboración propia del equipo. El diagrama representa la separación entre frontend, backend y PostgreSQL.*

## 5.1 Componentes principales

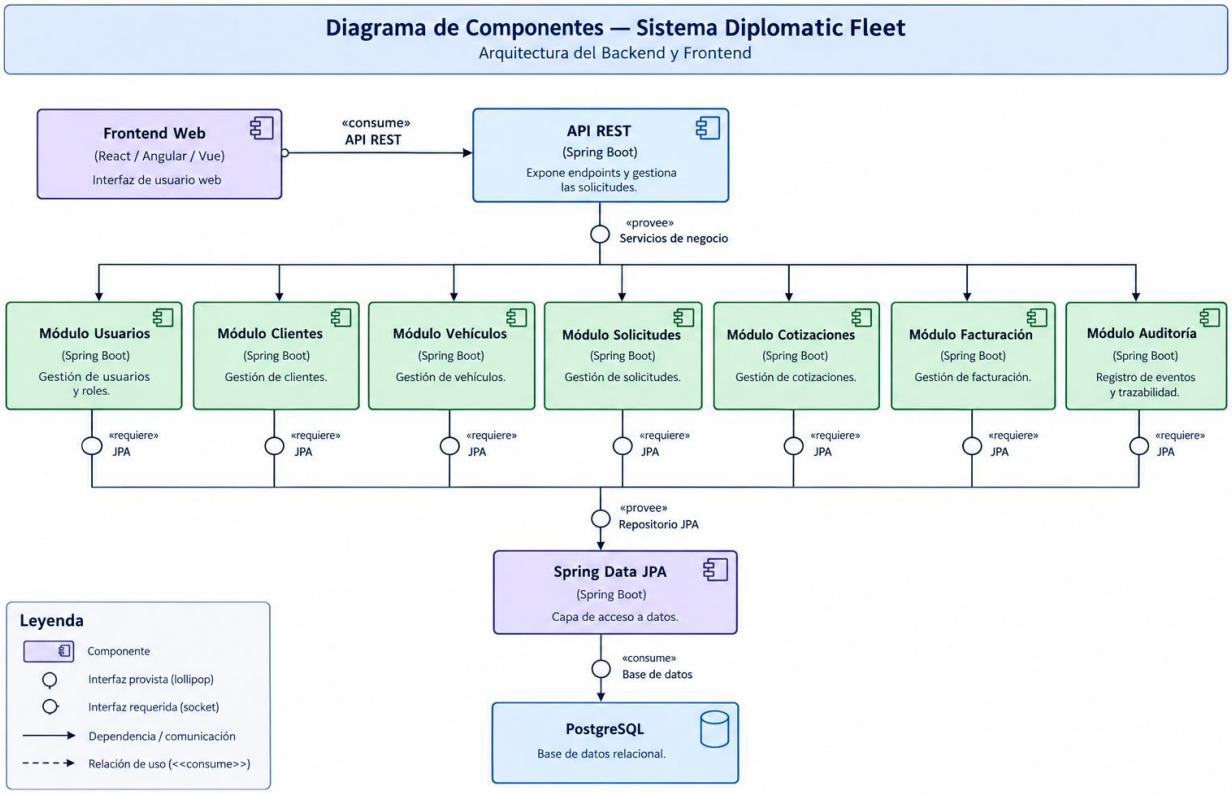



**Figura 2. Diagrama de componentes.**

*Nota. Elaboración propia del equipo. Se presentan las dependencias entre componentes de presentación, negocio y persistencia.*

## 5.2 Casos de uso

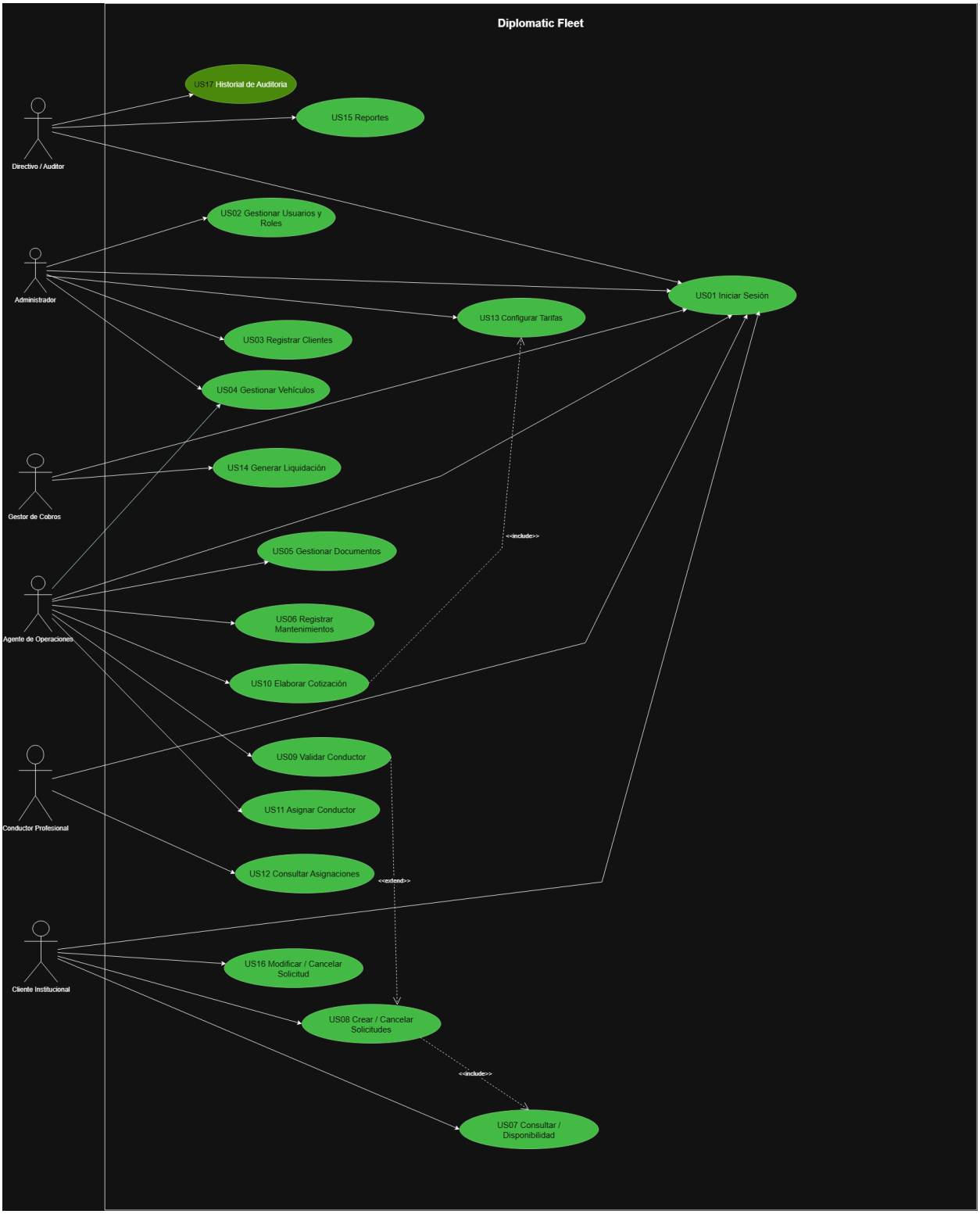



**Figura 3. Diagrama de casos de uso.**

*Nota. Elaboración propia del equipo. Los casos no implementados se consideran parte del trabajo futuro.*

### 5.3 Modelo de dominio

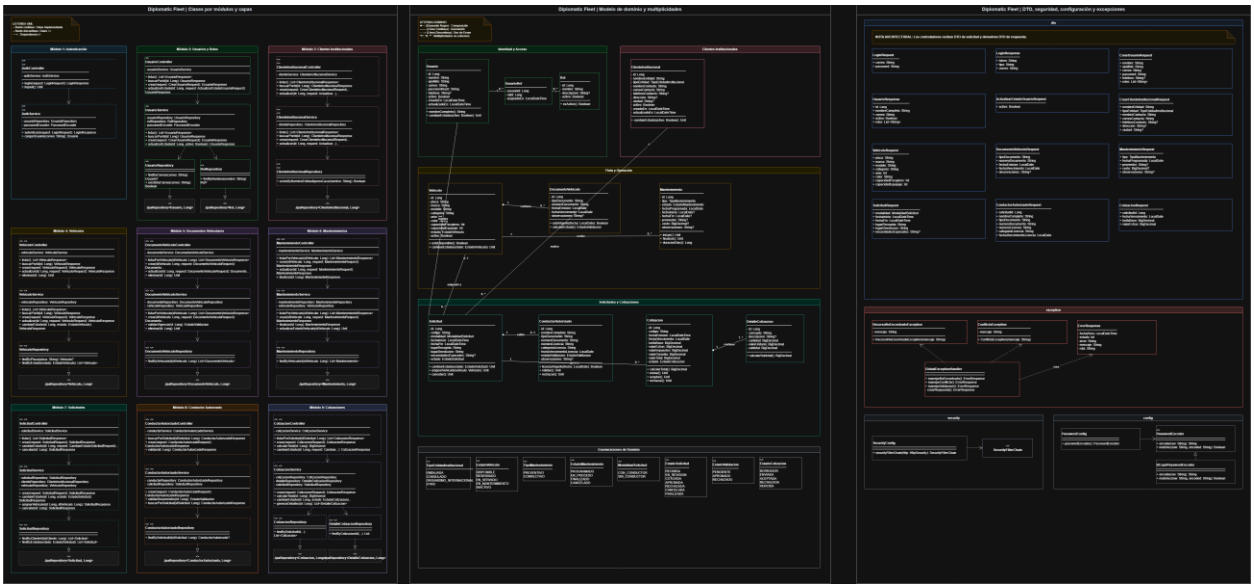


**Figura 4. Modelo UML del dominio.**

*Nota. Elaboración propia del equipo. Las entidades funcionales de Clientes y Vehículos se encuentran integradas con PostgreSQL.*

## 5.4 Secuencia

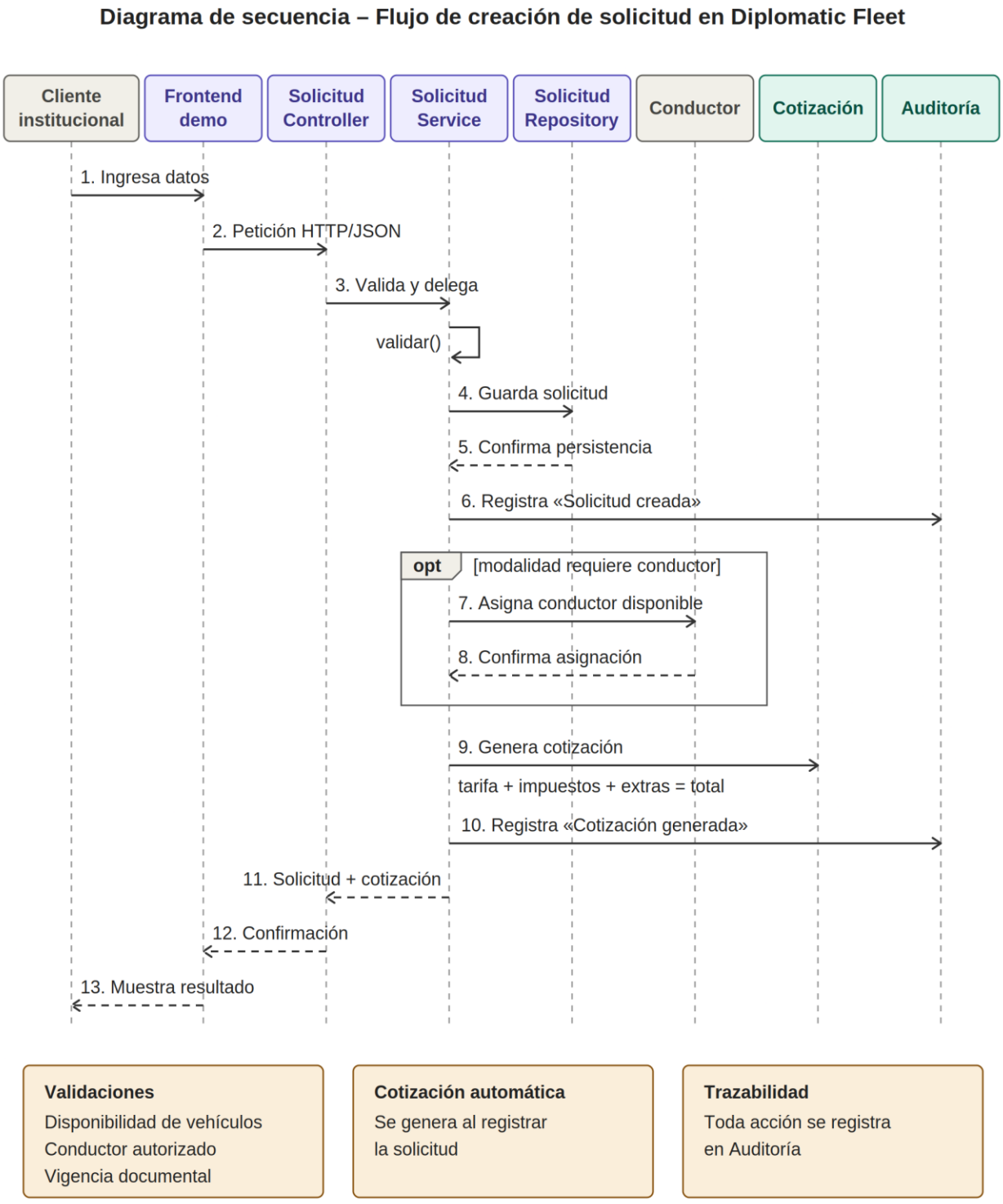



**Figura 5. Diagrama de secuencia del flujo principal.**

*Nota. Elaboración propia del equipo. El flujo ilustra la interacción entre la interfaz, la API y la persistencia.*

### 5.5 Estados y paquetes

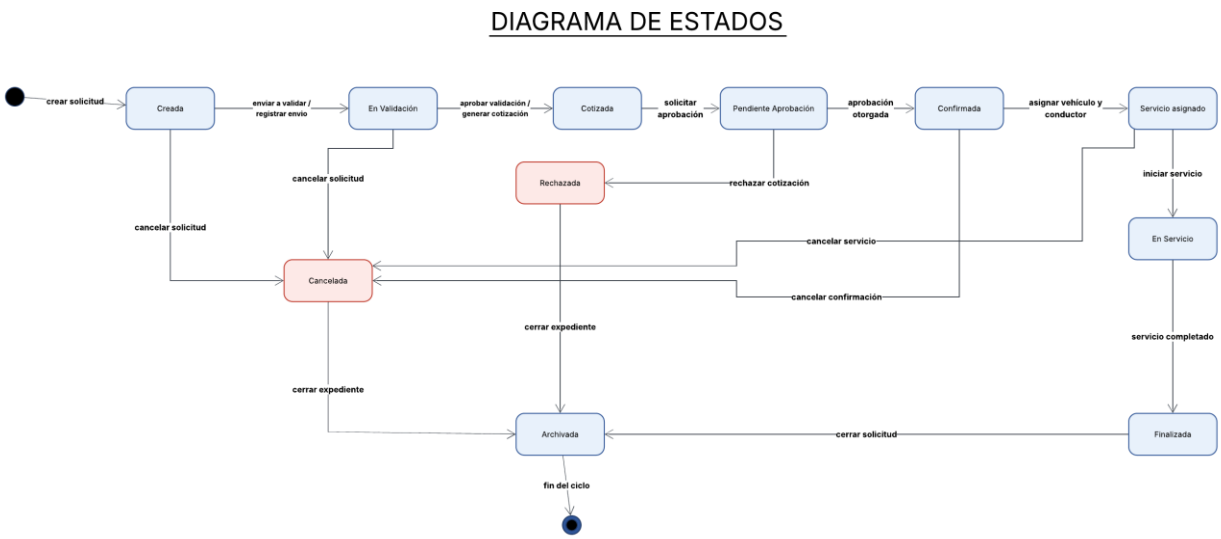



**Figura 6. Diagrama de estados.**

*Nota. Elaboración propia del equipo. Las transiciones completas corresponden al roadmap del producto.*

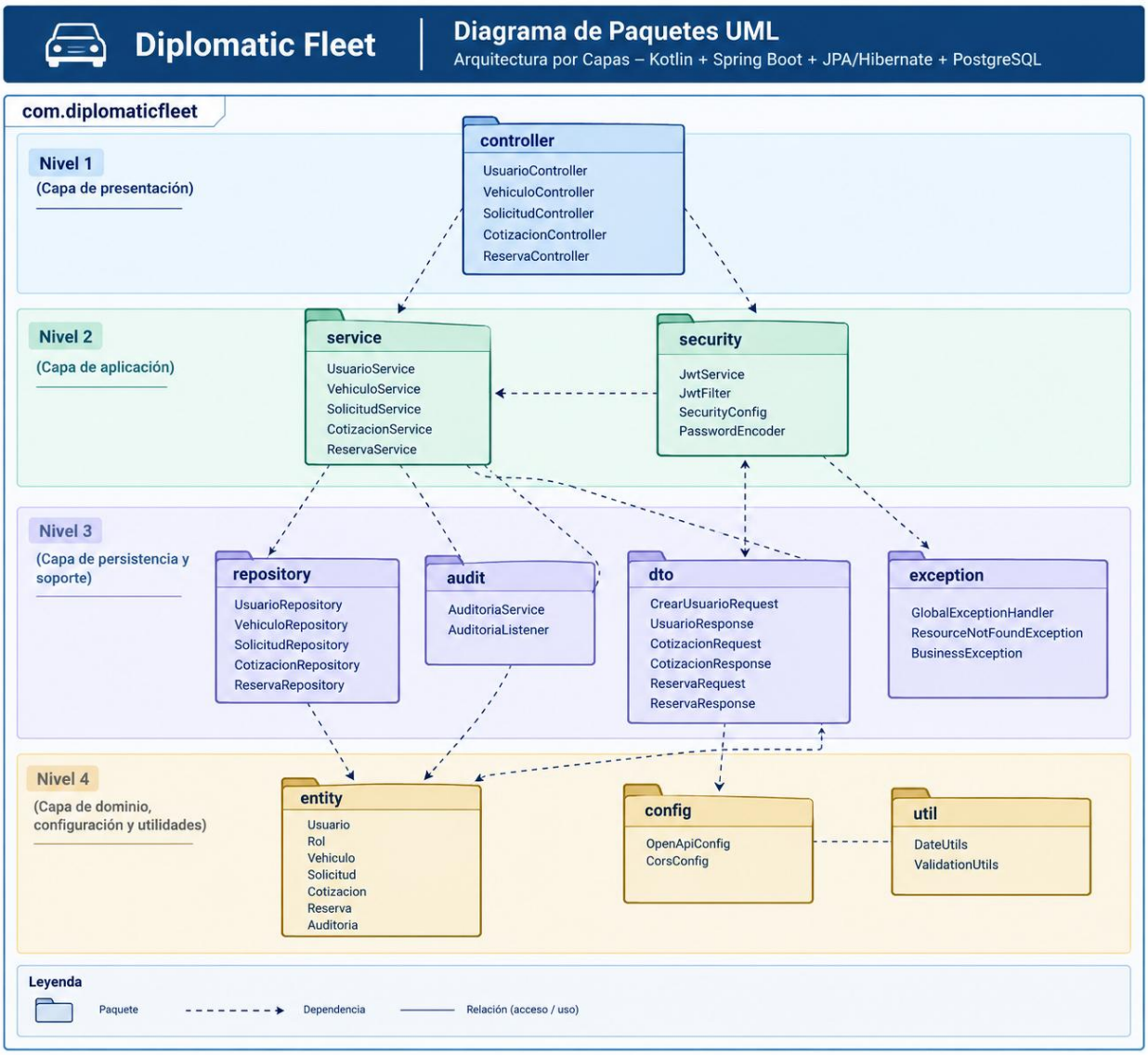



**Figura 7. Diagrama de paquetes.**

*Nota. Elaboración propia del equipo. La organización por módulos favorece el mantenimiento incremental.*

## 6. Arquitectura del código fuente

El backend organiza cada dominio por paquetes de responsabilidad. El frontend separa vistas, componentes compartidos, servicios HTTP, enrutamiento y estilos. Esta estructura reduce el acoplamiento y facilita aplicar en nuevos módulos el mismo patrón de Clientes y Vehículos.

| Capa o carpeta | Elementos actuales | Responsabilidad |
|---|---|---|

| | | |
|---|---|---|
| controller | VehiculoController, ClienteInstitucionalController | Exponer endpoints y delegar en servicios. |
| service | VehiculoService, ClienteInstitucionalService | Aplicar reglas, normalización, transacciones y mapeos. |
| repository | Repositorios JPA de vehículos, categorías y clientes | Abstraer consultas y persistencia. |
| domain / entity | Vehículo, Categoría y ClienteInstitucional | Mapear el dominio a PostgreSQL. |
| dto | Requests y Responses | Validar entradas y controlar las salidas públicas. |
| exception | GlobalExceptionHandler y excepciones de negocio | Uniformar respuestas 400, 404 y 409. |
| config | SecurityConfig | Proteger la API y configurar HTTP Basic temporal. |
| frontend/src/services | vehiculoService.ts y clienteService.ts | Centralizar solicitudes HTTP y manejo de errores. |
| frontend/src/views | LoginView, VehiculosView, ClientesView y vistas demo | Presentar formularios, indicadores, listados y navegación. |
| frontend/src/components/layout | DemoSidebar y DemoLayout | Compartir navegación y composición visual. |

### 6.1 Documentación del código

Las clases, interfaces, DTO y operaciones relevantes incorporan KDoc o comentarios TypeScript. La documentación explica propósito, parámetros, retornos, excepciones y reglas no evidentes. Se evita repetir literalmente el código y se priorizan decisiones como normalización, unicidad, cambios de estado y tratamiento de errores.

## 7. Modelo de datos

Las migraciones de Flyway crean las tablas, restricciones e índices. En la primera entrega se verificó la persistencia de clientes y vehículos. Las fechas de auditoría se mantienen en

OffsetDateTime y los campos únicos se validan en la capa de servicio antes de que PostgreSQL aplique sus propias restricciones.

### 7.1 Clientes institucionales

| Campo | Regla principal |
|---|---|
| nombre_entidad | Obligatorio, máximo 150 caracteres y único sin distinguir mayúsculas. |
| tipo_entidad | EMBAJADA, CONSULADO, ORGANISMO_INTERNACIONAL u OTRO. |
| numero_identificacion_fiscal | Opcional, máximo 40 caracteres y único cuando existe. |
| nombre_contacto | Obligatorio, máximo 140 caracteres. |
| correo_contacto | Obligatorio, formato válido, máximo 150 caracteres y único. |
| telefono_contacto | Opcional, máximo 30 caracteres. |
| direccion y ciudad | Opcionales, con límites de longitud. |
| activo | Permite inactivar sin eliminar el historial. |
| creado_en y actualizado_en | Auditoría temporal del registro. |

### 7.2 Vehículos

El módulo de vehículos conserva placa, categoría, marca, modelo, año, color, combustible, capacidad de pasajeros, capacidad de equipaje, estado, indicador activo y fechas de auditoría. La placa se normaliza y se valida su unicidad. La categoría debe existir y estar activa. El estado inicial utilizado en las pruebas fue DISPONIBLE.

## 8. Requisitos funcionales e historias de usuario

| ID | Descripción | Estado actual |
|---|---|---|
| US01 | Iniciar sesión según credenciales de demostración. | Funcional temporal |
| US02 | Gestionar usuarios y roles. | Prototipo navegable |
| US03 | Registrar y administrar clientes institucionales. | Funcional |
| US04 | Registrar y consultar vehículos. | Funcional |

| | | |
|---|---|---|
| US05 | Gestionar documentos y vigencias. | Prototipo navegable |
| US06 | Registrar mantenimientos. | Prototipo navegable |
| US07 | Consultar disponibilidad. | Prototipo navegable |
| US08 | Crear y consultar solicitudes. | Prototipo navegable |
| US09 | Validar conductor autorizado. | Prototipo navegable |
| US10 | Elaborar cotizaciones. | Prototipo navegable |

Las historias posteriores relacionadas con asignación de conductor profesional, tarifas, liquidación, reportes, auditoría y operaciones avanzadas permanecen en el backlog. La primera entrega priorizó la base arquitectónica y dos módulos completos de extremo a extremo.

## 9. Módulos implementados

### 9.1 Acceso y sesión de demostración

El formulario de acceso valida el formato del correo, una longitud mínima de contraseña y la aceptación de términos y política de privacidad. El correo temporal autorizado es admin@diplomaticfleet.co. La contraseña se valida realizando una solicitud protegida al backend. Si la autenticación falla, las credenciales se eliminan de sessionStorage y la navegación no continúa.

El encabezado X-Requested-With identifica las solicitudes del frontend y evita que el navegador muestre el cuadro nativo de HTTP Basic ante un error 401. La sesión se mantiene únicamente durante la pestaña y se elimina al cerrar sesión.

### 9.2 Clientes institucionales

El módulo permite registrar, listar, buscar, consultar y actualizar clientes institucionales. El frontend muestra indicadores de total, activos, inactivos y clasificaciones utilizadas. El botón Activar o Inactivar ejecuta PUT /api/clientes/{id}, por lo que el cambio permanece después de actualizar la página.

| Endpoint | Método | Resultado |
|---|---|---|
| /api/clientes | POST | Registra una institución y devuelve 201 Created. |
| /api/clientes | GET | Lista instituciones registradas. |
| /api/clientes/{id} | GET | Consulta una institución o devuelve 404. |
| /api/clientes/{id} | PUT | Actualiza datos o estado y devuelve 200. |

### 9.3 Vehículos

El módulo permite registrar y listar vehículos, filtrar por placa, marca, modelo, categoría, estado o combustible, y visualizar indicadores operativos. La creación responde con 201 Created y el listado se recupera mediante GET /api/vehiculos. El formulario normaliza la placa y los textos antes de enviarlos.

| Endpoint | Método | Resultado |
|---|---|---|
| /api/vehiculos | POST | Registra un vehículo y devuelve 201 Created. |
| /api/vehiculos | GET | Lista vehículos registrados. |
| /api/vehiculos/{id} | GET | Consulta un vehículo o devuelve 404. |
| /api/vehiculos/{id} | PUT | Actualiza la información del vehículo. |

## 10. Módulos navegables en fase de integración

La barra lateral compartida conserva diez módulos visibles en todas las vistas, incluso al entrar en Vehículos. Esta corrección evita perder el acceso al resto del sistema. DemoSidebar.vue identifica la ruta activa y DemoLayout.vue proporciona la composición común para las vistas demostrativas.

- Inicio: resumen general del sistema.
- Documentos: control futuro de documentos vehiculares y vigencias.
- Mantenimientos: programación e historial preventivo y correctivo.
- Disponibilidad: consulta futura por periodo, categoría y capacidad.

• Solicitudes: registro y seguimiento de servicios institucionales.

• Validación de conductor: comprobación futura de licencia y requisitos.

• Cotizaciones: cálculo y seguimiento de propuestas comerciales.

• Usuarios y roles: administración futura de cuentas, perfiles y permisos.

Estas vistas no sustituyen una implementación funcional. Su propósito es representar el alcance, validar la navegación y servir como base visual para integrar API y persistencia en iteraciones posteriores.

## 11. Seguridad y control de acceso

La primera entrega utiliza Spring Security con HTTP Basic y credenciales técnicas suministradas mediante variables de entorno. La configuración protege los endpoints de la API y excluye las rutas REST del control CSRF durante la integración local con Vue.js. Esta solución es temporal y no debe interpretarse como el mecanismo definitivo de producción.

| Control | Aplicación actual |
|---|---|
| Secretos | La contraseña de PostgreSQL y la contraseña técnica se configuran en la terminal y no se versionan. |
| Credenciales del navegador | Se almacenan temporalmente en sessionStorage. |
| Respuesta 401 | Se procesa dentro de la interfaz sin cuadro nativo del navegador. |
| Correo de demo | Validación temporal con admin@diplomaticfleet.co. |
| Roles reales | Pendientes de conexión con usuarios almacenados en PostgreSQL. |
| Contraseñas productivas | Se proyecta almacenamiento con BCrypt y flujo definitivo de autenticación. |

## 12. Validaciones y manejo de errores

GlobalExceptionHandler centraliza errores de validación y negocio para que el frontend reciba una estructura uniforme. ConflictoException representa duplicados o incompatibilidades y RecursoNoEncontradoException representa consultas de recursos inexistentes.

| Código | Uso |
|---|---|
| 400 Bad Request | Datos obligatorios ausentes, formatos inválidos o restricciones de longitud. |
| 401 Unauthorized | Credenciales inválidas o sesión no disponible. |
| 404 Not Found | Cliente, vehículo o recurso solicitado inexistente. |
| 409 Conflict | Nombre, correo, placa o identificación fiscal duplicados. |

Los servicios TypeScript intentan extraer mensajes de validación del cuerpo JSON. Cuando la respuesta no contiene JSON, muestran el código HTTP. Esta estrategia permite mantener mensajes comprensibles sin duplicar reglas de negocio en Vue.

## 13. Pruebas y evidencias de calidad

El equipo ejecutó pruebas del backend y compilaciones del frontend después de cada bloque importante. La prueba de contexto confirmó que Spring Boot puede inicializar la aplicación con PostgreSQL y Flyway. Las compilaciones de Vue validaron TypeScript y generaron los recursos de producción.

| Prueba o verificación | Resultado documentado |
|---|---|
| .\gradlew.bat compileKotlin | BUILD SUCCESSFUL para entidades, repositorios, DTO, servicios y controladores. |
| .\gradlew.bat test | BUILD SUCCESSFUL con carga del contexto y cierre correcto del datasource. |
| npm run build | Validación TypeScript y generación de producción completadas. |
| POST /api/vehiculos | HTTP 201 y persistencia del vehículo. |
| GET /api/vehiculos | Listado real recuperado desde PostgreSQL. |
| Clientes desde Vue | Registro, búsqueda, activación e inactivación persistentes. |
| Acceso erróneo | Correo o contraseña incorrectos rechazados. |
| Acceso autorizado | Navegación a /inicio con credenciales de demo. |

| | |
|---|---|
| Git | Main sincronizada y árbol de trabajo limpio después de los merges. |

Las pruebas actuales constituyen evidencia de integración y compilación, pero no sustituyen una cobertura completa de pruebas unitarias y de integración. Se requieren pruebas automatizadas específicas para duplicados, cambios de estado, autorización por rol, disponibilidad y cotizaciones.

## 14. Instalación y ejecución local

### 14.1 Requisitos

- Git.
- Java JDK 21.
- PostgreSQL y una base denominada diplomatic_fleet.
- Node.js compatible con el package.json del frontend.
- Conexión a Internet para la descarga inicial de dependencias.
- Puertos 8080, 5173 y 5432 disponibles.

### 14.2 Backend

Desde la raíz del repositorio se configuran DB_URL, DB_USERNAME, DB_PASSWORD, SPRING_SECURITY_USER_NAME y SPRING_SECURITY_USER_PASSWORD en la terminal actual. Luego se ejecutan las pruebas y bootRun. Flyway aplica las migraciones al iniciar.

Comandos de referencia:

- .\gradlew.bat test
- .\gradlew.bat bootRun

### 14.3 Frontend

Dentro de frontend se ejecuta npm install una vez por equipo, npm run build para validar y npm run dev para iniciar Vite. Durante el desarrollo, Vite redirige /api hacia http://localhost:8080 mediante el proxy definido en vite.config.ts.

- npm install
- npm run build
- npm run dev

## 15. Traslado del sistema a otro equipo

Para presentar desde la laptop de otro integrante se debe clonar el repositorio desde GitHub, instalar dependencias, crear una base PostgreSQL vacía y configurar variables de entorno locales. Git transfiere el código y las migraciones, pero no copia los registros existentes en la base de datos del primer equipo.

1. Clonar: git clone https://github.com/Riiperk/Diplomatic-Fleet.git.
2. Entrar al repositorio, cambiar a main y ejecutar git pull origin main.
3. Crear la base diplomatic_fleet en PostgreSQL.
4. Ejecutar npm install y npm run build en frontend.
5. Configurar las variables del backend y ejecutar .\gradlew.bat test.
6. Iniciar backend y frontend en terminales separadas.
7. Registrar nuevamente los datos de demostración porque PostgreSQL es local a cada equipo.

## 16. Resultados de la primera entrega

- Se dispone de una aplicación web local con backend y frontend separados.
- Clientes y Vehículos completan el recorrido Vue.js, Spring Boot, JPA y PostgreSQL.

• La navegación común expone diez módulos con diseño responsive.

• El acceso rechaza credenciales incorrectas y conserva una sesión temporal controlada.

• Los errores 400, 404 y 409 se manejan de manera uniforme.

• El repositorio conserva ramas, commits individuales, revisiones y documentación.

• La aplicación puede clonarse y ejecutarse en otro equipo mediante migraciones reproducibles.

## 17. Limitaciones y trabajo futuro

• Reemplazar el correo fijo y HTTP Basic por usuarios, roles y sesiones reales.

• Implementar lógica y persistencia de Documentos, Mantenimientos, Disponibilidad, Solicitudes, Validación de conductor y Cotizaciones.

• Agregar edición completa y paginación en Clientes y Vehículos.

• Asociar solicitudes con clientes y vehículos disponibles.

• Implementar autorización por rol y auditoría de acciones.

• Ampliar las pruebas unitarias, de integración y extremo a extremo.

• Optimizar el recurso gráfico principal del login, cuyo tamaño de compilación es superior al resto de activos.

• Preparar despliegue del backend y PostgreSQL en un entorno accesible para una demo pública.

## 18. Conclusiones

La primera entrega establece una base técnica funcional y verificable para Diplomatic Fleet. El proyecto ya demuestra que el frontend puede consumir una API protegida, aplicar validaciones, almacenar información institucional y vehicular en PostgreSQL y mantener una navegación coherente entre módulos.

La separación por capas y módulos reduce el riesgo de crecimiento desordenado. La implementación de Clientes y Vehículos sirve como patrón para continuar el resto del alcance. La trazabilidad de GitHub, la documentación del código, las pruebas de compilación y las migraciones reproducibles fortalecen la calidad de la entrega.

El estado actual debe presentarse como desarrollo incremental. Dos módulos se encuentran conectados de extremo a extremo y ocho módulos adicionales representan la interfaz y el roadmap de integración. Esta distinción permite explicar con precisión tanto los logros como el trabajo pendiente.

## Referencias

American Psychological Association. (2025). Student paper setup guide (7th ed.). https://apastyle.apa.org/instructional-aids/student-paper-setup-guide.pdf

Equipo Diplomatic Fleet. (2026). Manual integral corregido de Diplomatic Fleet [Documento académico no publicado].

GitHub. (2026). Cloning a repository. https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

PostgreSQL Global Development Group. (2026). PostgreSQL documentation. https://www.postgresql.org/docs/

Spring. (2026). Spring Boot reference documentation. https://docs.spring.io/spring-boot/reference/

Spring. (2026). Spring Data JPA. https://spring.io/projects/spring-data-jpa/

Vue.js. (2026). Vue.js documentation. https://vuejs.org/guide/

Vue Router. (2026). Vue Router documentation. https://router.vuejs.org/

## Apéndice A. Credenciales y datos de demostración

| Elemento | Valor de demostración |
|---|---|
| Correo | admin@diplomaticfleet.co |
| Contraseña | Demo1234 |
| Usuario técnico HTTP Basic | user |
| Base de datos local | diplomatic_fleet |
| Backend | http://localhost:8080 |
| Frontend Vite | http://localhost:5173 |

Estas credenciales son exclusivamente académicas y no deben reutilizarse en producción. La contraseña de PostgreSQL no se documenta y debe configurarse de forma local mediante una variable de entorno.

## Apéndice B. Lista de verificación para la exposición

- Main actualizada y árbol de trabajo limpio.
- PostgreSQL activo y base diplomatic_fleet disponible.
- Backend iniciado y puerto 8080 libre.
- Frontend iniciado y puerto 5173 libre.
- Acceso válido comprobado.
- Dos clientes institucionales formales preparados.
- Dos vehículos preparados.
- Registro, búsqueda y cambio de estado demostrados.
- Módulos prototipo presentados como trabajo futuro.
- Secretos y contraseñas reales ausentes del repositorio y del documento.

## Apéndice C. Control documental

| Campo | Información |
|---|---|
| Documento | Documentación técnica y funcional de la primera entrega |
| Proyecto | Diplomatic Fleet |
| Versión | 3.0 |
| Estado | Entrega académica preparada para presentación |
| Fecha | Octubre de 2026 |
| Repositorio | https://github.com/Riiperk/Diplomatic-Fleet |
| Formato | APA 7.ª edición, documento de estudiante |
| Equipo | Daniela Largo, Xiomara Lobatón, Bryan Steven Pardo y María Camila Pacheco |